

realtimeshadow 报告

安燕杰李志阳

2026 年 5 月 29 日

摘 要

本文围绕 WebGL 实时阴影渲染任务，基于 Two-Pass Shadow Map 框架依次实现硬阴影、PCF 软阴影和 PCSS 软阴影。系统首先从光源视角生成 Shadow Map，再在相机视角中将片元变换到光源空间并进行深度比较。随后在硬阴影基础上引入 Poisson 圆盘采样，实现固定滤波半径的 PCF；最后进一步实现 blocker search、半影估计和动态滤波半径，得到 PCSS 阴影效果。实验结果表明，硬阴影边缘清晰但锯齿明显，PCF 能够有效平滑边界，PCSS 则能够在接触区域保持较硬阴影，并在远离遮挡物处产生更宽的软阴影过渡。

1 问题背景

实时阴影是交互式图形系统中的基础问题。对于三维场景而言，如果只使用 Blinn-Phong 等局部光照模型，物体之间不会产生遮挡关系，渲染结果会缺少空间层次感。阴影能够表达光源、遮挡物和接收面的相对位置，因此对增强真实感非常重要。

离线渲染中可以通过路径追踪或光线追踪获得较高质量的软阴影，但这些方法代价较高，不适合本项目所采用的 WebGL 实时渲染框架。本次任务采用 Shadow Map 作为基础方法。Shadow Map 的核心思想是先从光源视角记录场景深度，再从相机视角渲染时查询该深度图，从而判断当前片元是否被其他物体遮挡。

基础 Shadow Map 可以得到硬阴影，但它有两个典型问题。第一，Shadow Map 分辨率有限，边缘容易出现锯齿。第二，硬阴影不能表达面积光源下真实存在的半影区域。为解决这两个问题，本项目进一步实现 PCF (Percentage Closer Filtering) 和 PCSS (Percentage Closer Soft Shadow)。PCF 通过多次深度比较并求平均，得到柔化边缘；PCSS 则在 PCF 之前估计遮挡物与接收面的深度关系，使滤波半径随半影大小动态变化。

2 研究内容与方法

2.1 整体渲染流程

本项目采用 Two-Pass Shadow Map。第一遍是 Shadow Pass，从光源视角渲染场景，将深度写入 FBO 对应的纹理中；第二遍是 Camera Pass，从相机视角正常渲染模型，并在片元着色器中使用 Shadow Map 判断可见性。

项目中普通渲染网格保存在 `renderer.meshes`，阴影渲染网格保存在 `renderer.shadowMeshes`。加载 OBJ 模型时，每个 mesh 同时创建 `PhongMaterial` 和 `ShadowMaterial`，前者用于 Camera Pass，

后者用于 Shadow Pass。渲染循环中先绑定光源 FBO 并清空，再绘制所有 shadow mesh；随后回到默认帧缓冲，绘制相机视角下的普通 mesh。

Listing 1 Shadow Pass 与 Camera Pass 的主要流程

```
1 // Shadow pass
2 gl.bindFramebuffer(gl.FRAMEBUFFER, this.lights[l].entity.fbo);
3 gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
4 for (let i = 0; i < this.shadowMeshes.length; i++) {
5     this.shadowMeshes[i].draw(this.camera);
6 }
7
8 // Camera pass
9 for (let i = 0; i < this.meshes.length; i++) {
10     this.gl.useProgram(this.meshes[i].shader.program.glShaderProgram);
11     this.gl.uniform3fv(
12         this.meshes[i].shader.program.uniforms.uLightPos,
13         this.lights[l].entity.lightPos
14     );
15     this.meshes[i].draw(this.camera);
16 }
```

2.2 光源空间矩阵与 Shadow Map

为了将模型顶点变换到光源视角，需要计算光源 MVP 矩阵：

$$MVP_{light} = P_{light}V_{light}M$$

其中，模型矩阵由平移和缩放构成，视图矩阵由 `lookAt(lightPos, focalPoint, lightUp)` 构成。由于本项目使用方向光，投影矩阵采用正交投影。当前实现中光源位置为 (0, 80, 80)，光源看向场景中心，正交投影范围根据 Mary 模型和地板范围进行收紧：

Listing 2 当前光源 MVP 的关键实现

```
1 mat4.identity(modelMatrix);
2 mat4.translate(modelMatrix, modelMatrix, translate);
3 mat4.scale(modelMatrix, modelMatrix, scale);
4
5 mat4.lookAt(viewMatrix, this.lightPos, this.focalPoint, this.lightUp);
6 mat4.ortho(projectionMatrix, -120, 120, -70, 110, 40, 240);
7
8 mat4.multiply(lightMVP, projectionMatrix, viewMatrix);
9 mat4.multiply(lightMVP, lightMVP, modelMatrix);
```

Shadow Pass 的片元着色器将 `gl_FragCoord.z` 打包到 RGBA 纹理中。Camera Pass 中再通过 `unpack()` 还原深度值。由于 WebGL1 中直接使用深度纹理并不总是方便，使用 RGBA 编码深度是一种兼容性较好的实现方式。

2.3 硬阴影实现

硬阴影阶段的核心函数为 `useShadowMap()`。片元首先将 `vPositionFromLight` 做透视除法，并从 NDC 的 $[-1, 1]$ 映射到纹理坐标 $[0, 1]$ 。随后用当前片元深度和 Shadow Map 中记录的最近深度进行比较：

$$visibility = \begin{cases} 0, & z_{receiver} - bias > z_{shadow} \\ 1, & otherwise \end{cases}$$

当前实现使用统一的 `SHADOW_BIAS = 0.004`，用于缓解 shadow acne。若片元投影坐标超出 Shadow Map 范围，则直接认为可见，避免边界外错误采样。

2.4 PCF 软阴影实现

硬阴影只做一次深度比较，结果是二值的，因此边界容易锯齿。PCF 的方法是在片元周围取多个采样点，每个采样点独立执行 shadow test，最后对所有结果取平均：

$$visibility = \frac{1}{N} \sum_{i=1}^N shadowTest(uv + offset_i \cdot r)$$

本项目使用 Poisson 圆盘采样生成不规则但相对均匀的采样核。当前采样数为 40，PCF 固定滤波半径为 `PCF_FILTER_SIZE = 0.0035`。为了避免采样偏移越界导致纹理环绕，FBO 中的 Shadow Map 纹理设置为 `CLAMP_TO_EDGE`，并且在 shader 中对每个 `sampleUV` 单独做边界检查。

Listing 3 固定半径 PCF 的复用接口

```

1 float PCFWithFilterSize(sampler2D shadowMap, vec4 coords, float filterSize) {
2     ...
3     for (int i = 0; i < PCF_NUM_SAMPLES; i++) {
4         vec2 offset = poissonDisk[i] * filterSize;
5         vec2 sampleUV = projCoords.xy + offset;
6         ...
7         visibility += currentDepth - bias > closestDepth ? 0.0 : 1.0;
8     }
9     return visibility / float(PCF_NUM_SAMPLES);
10 }
11
12 float PCF(sampler2D shadowMap, vec4 coords) {
13     return PCFWithFilterSize(shadowMap, coords, PCF_FILTER_SIZE);
14 }

```

2.5 PCSS 软阴影实现

PCSS 在 PCF 基础上增加了遮挡物搜索和半影估计。首先，`findBlocker()` 在当前片元的 Shadow Map 坐标附近搜索比接收点更靠近光源的样本。如果某个样本满足：

$$z_{receiver} - bias > z_{sample}$$

则认为该样本对应 blocker。函数累加所有 blocker 的深度并取平均，得到 avgBlockerDepth。如果没有找到 blocker，则当前片元不在阴影中，直接返回 1.0。

Listing 4 blocker 搜索的关键逻辑

```

1 float findBlocker(sampler2D shadowMap, vec2 uv, float zReceiver) {
2     poissonDiskSamples(uv);
3     float blockerDepthSum = 0.0;
4     int blockerCount = 0;
5     float bias = SHADOW_BIAS;
6
7     for (int i = 0; i < BLOCKER_SEARCH_NUM_SAMPLES; i++) {
8         vec2 sampleUV = uv + poissonDisk[i] * BLOCKER_SEARCH_SIZE;
9         float closestDepth = unpack(texture2D(shadowMap, sampleUV));
10        if (zReceiver - bias > closestDepth) {
11            blockerDepthSum += closestDepth;
12            blockerCount++;
13        }
14    }
15
16    if (blockerCount == 0) {
17        return -1.0;
18    }
19    return blockerDepthSum / float(blockerCount);
20 }

```

得到平均 blocker 深度后，PCSS 根据接收点深度和 blocker 深度估算半影大小：

$$penumbraRatio = \frac{z_{receiver} - z_{blocker}}{\max(z_{blocker}, \epsilon)}$$

再乘以模拟光源大小 LIGHT_SIZE_UV 得到动态滤波半径。当前参数将滤波半径限制在 [0.0008, 0.009] 范围内，避免阴影过硬或过糊。最终调用 PCFWithFilterSize() 完成动态半径的多采样过滤。

Listing 5 PCSS 的三步实现

```

1 float PCSS(sampler2D shadowMap, vec4 coords) {
2     vec3 projCoords = coords.xyz;
3
4     float avgBlockerDepth = findBlocker(shadowMap, projCoords.xy, projCoords.z);
5     if (avgBlockerDepth < 0.0) {
6         return 1.0;
7     }
8
9     float penumbraRatio =
10         (projCoords.z - avgBlockerDepth) / max(avgBlockerDepth, EPS);
11     float filterSize = clamp(
12         penumbraRatio * LIGHT_SIZE_UV,
13         PCSS_MIN_FILTER_SIZE,
14         PCSS_MAX_FILTER_SIZE
15     );
16
17     return PCFWithFilterSize(shadowMap, coords, filterSize);

```

18 }

2.6 光照合成方式

为了避免阴影区域被整体压黑，当前实现没有直接将 `visibility` 乘到最终颜色上，而是只让它影响直接光照项：

Listing 6 阴影只衰减直接光照

```
1 vec3 radiance = ambient + visibility * (diffuse + specular);
2 vec3 phongColor = pow(radiance, vec3(1.0 / 2.2));
```

这样阴影区域仍保留环境光贡献，视觉上比直接 `phongColor * visibility` 更自然。

2.7 主要参数

表 1 当前实现使用的主要参数

参数	当前值	作用
Shadow Map 分辨率	8192 × 8192	提高阴影采样精度
光源位置	(0, 80, 80)	方向光视角位置
光源正交投影	[-120, 120], [-70, 110], 40, 240	覆盖当前场景并控制深度精度
NUM_SAMPLES	40	PCF 与 PCSS 的采样数
SHADOW_BIAS	0.004	缓解 shadow acne
PCF_FILTER_SIZE	0.0035	PCF 固定滤波半径
BLOCKER_SEARCH_SIZE	0.0045	PCSS blocker 搜索半径
LIGHT_SIZE_UV	0.007	模拟面积光大小
PCSS_MIN_FILTER_SIZE	0.0008	PCSS 最小滤波半径
PCSS_MAX_FILTER_SIZE	0.009	PCSS 最大滤波半径

3 实验结果与分析

实验场景由两个 Mary 模型和地板组成。光源位于 (0, 80, 80)，相机使用 OrbitControls 进行观察。实验中分别切换硬阴影、PCF 和 PCSS 三种模式，并保存结果到 `images` 文件夹。

3.1 硬阴影结果

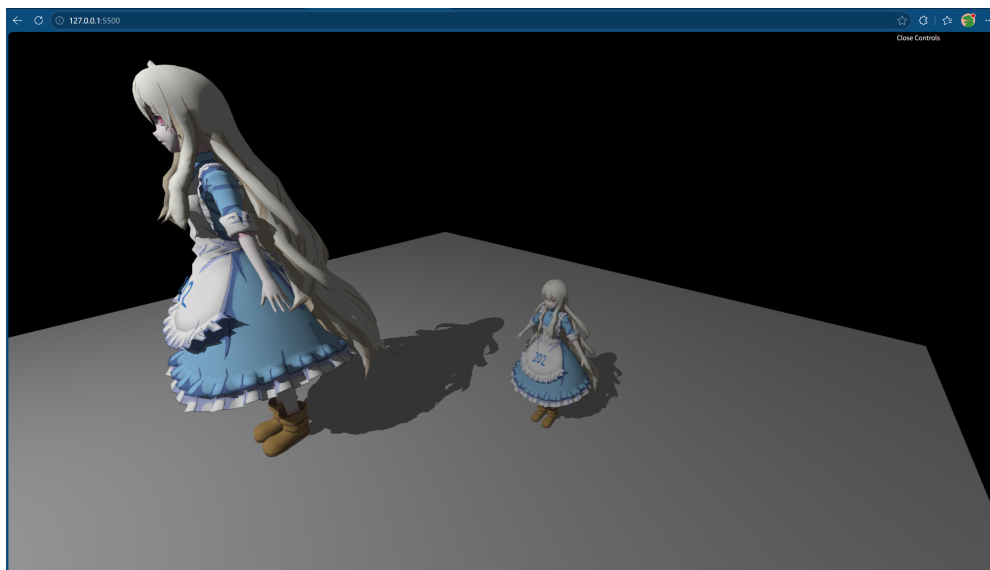


图 1 Shadow Map 硬阴影结果

图 1 展示了基础 Shadow Map 的结果。可以看到物体能够在地板上产生明确阴影，说明光源 MVP、Shadow Pass、深度打包与深度比较均已正常工作。但是硬阴影只进行一次深度比较，边界是二值变化，因此在模型轮廓附近容易出现锯齿和较硬的边缘。这种结果适合验证 Shadow Map 的正确性，但真实感不足。

3.2 PCF 结果

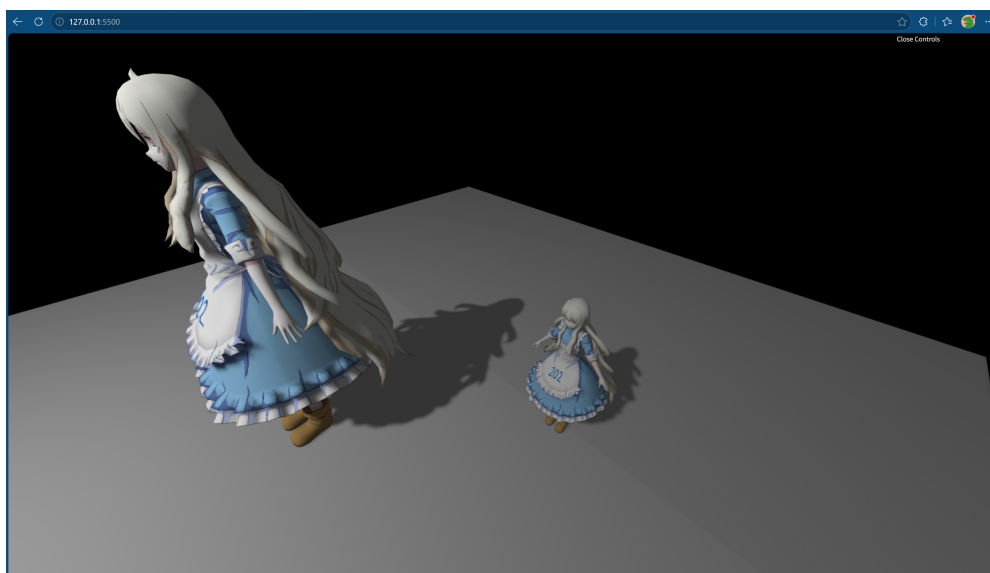


图 2 PCF 软阴影结果

图 2 展示了固定半径 PCF 的结果。与硬阴影相比，PCF 通过多次采样和平均明显削弱了边界锯齿，阴影边缘变成连续过渡。PCF 的实现简单稳定，适合在实时场景中使用。但是由于滤波半径固定，阴影的软硬程度与遮挡物和接收面的距离无关，接触区域和远离区域的边缘宽度变化不明显。

3.3 PCSS 结果

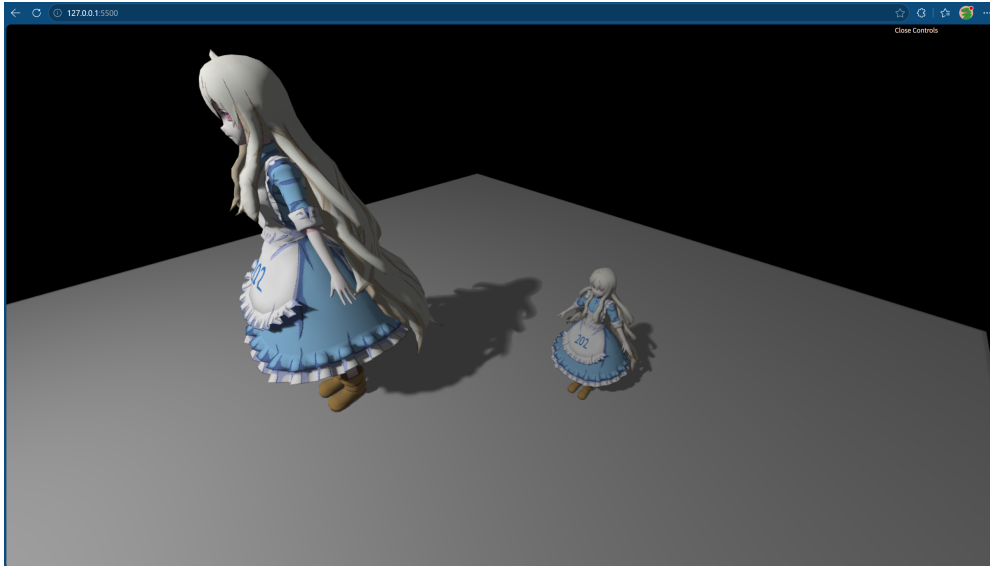


图 3 PCSS 软阴影结果

图 3 展示了 PCSS 的结果。PCSS 会先搜索 blocker 平均深度，再根据 receiver 与 blocker 的深度差估计半影大小。相比固定半径 PCF，PCSS 更符合面积光下的软阴影规律：遮挡物接近接收面的位置保持较清晰的接触阴影，而距离较远的位置滤波半径增大，边缘过渡更宽。

从实验观察看，PCSS 的视觉质量受采样数和参数影响较大。当采样数较小时，复杂模型轮廓处会出现颗粒感；当采样数过大时，WebGL 页面刷新和截图会明显变慢。最终采用 40 次采样作为质量与性能的折中。同时，LIGHT_SIZE_UV 和 PCSS_MAX_FILTER_SIZE 不能过大，否则阴影会过度模糊；SHADOW_BIAS 也需要保持适中，否则会在 shadow acne 和 Peter Panning 之间产生取舍。

3.4 三种方法对比

表 2 三种阴影方法的效果对比

方法	优点	缺点	本项目表现
Shadow Map	实现简单，阴影关系明确	边缘硬，锯齿明显	可验证遮挡关系正确
PCF	边缘平滑，实时性较好	滤波半径固定，缺少接触变化	适合稳定展示软阴影
PCSS	半影随深度变化，更接近面积光	开销较大，参数敏感	能表现接触硬、远处软

总体而言，硬阴影主要用于验证基础 Shadow Map 管线，PCF 解决了边界锯齿问题，PCSS 则进一步引入了深度相关的半影估计。由于当前项目运行在 WebGL1 环境，并使用 8192×8192 的 Shadow Map 和多次采样，PCSS 的性能压力较高，刷新时可能受浏览器缓存、WebGL context 和 GPU 资源释放时序影响。因此实际截图时需要在切换 shader 后强制刷新，并等待资源加载完成。

4 总结

本文基于 WebGL 实现了实时阴影渲染中的三个核心任务。首先，通过 Two-Pass Shadow Map 建立了从光源视角记录深度、从相机视角进行遮挡判断的基础管线，实现了硬阴影。随后，在硬阴影基础上引入 Poisson 圆盘采样，实现了固定半径 PCF，使阴影边缘由二值跳变变为连续过渡。最后，实现 PCSS 中的 blocker search、平均遮挡物深度计算、半影大小估计和动态半径 PCF，使阴影能够呈现接触处较硬、远离遮挡物处较软的变化。

在实现过程中，关键问题包括光源 MVP 的正确构造、Shadow Map 深度的打包与解包、采样越界处理、bias 参数选择以及性能与质量的平衡。最终实现中将 Shadow Map 纹理设置为 CLAMP_TO_EDGE，对 PCF/PCSS 采样点进行边界检查，并将阴影可见性只作用于直接光照项，从而避免环境光被整体压暗。

实验结果说明，Shadow Map、PCF 和 PCSS 三种方法均已在当前场景中完成并可通过 shader 注释切换。硬阴影用于展示基本遮挡关系，PCF 提供稳定的软化效果，PCSS 则进一步表现深度相关的软阴影特征。后续若继续改进，可以从更稳定的采样序列、降低 Shadow Map 显存开销、加入调试可视化以及使用时间累积降低噪声等方向展开。